Décrypter le Transformer : du caractère au concept

Mini-projet - MSO 3.7

Julien Velcin

Ecole Centrale de Lyon (2025-2026)

Ce mini projet vise à manipuler, évaluer et améliorer un modèle de langage génératif (LLM) basé sur l'architecture du Transformer (décodeur seul, type GPT). Il reprend une structure inspirée de *nanoGPT* (A. Karpathy) mais vous demande d'aller plus loin que le simple entraînement, en implémentant des techniques de génération avancées et en critiquant les performances.

1 Prise en main et entraînement robuste

La première étape consiste à récupérer le code de base vu en cours, choisir un corpus (Shakespeare, Molière, etc.), et lancer un premier apprentissage. Le modèle initial fonctionne au niveau des **caractères**.

Objectifs :

- **Exploration :** Faites tourner le code. Observez la baisse de la *loss*.
- **Checkpointing :** L'entraînement peut être long. Implémentez un mécanisme de sauvegarde (`torch.save`) et de chargement (`load_state_dict`) des poids du modèle pour pouvoir interrompre et reprendre l'apprentissage sans tout perdre.
- **Hyper-paramètres :** Testez l'impact de la taille du contexte (*block size*), du nombre de couches (*n_ layer*) et du nombre de têtes d'attention.
- **Prompting :** Influencez la génération en forçant le début de la phrase (le *context*) avant de laisser le modèle imaginer la suite.

2 Évaluations quantitatives

La *loss* d'entraînement ne suffit pas à juger la qualité d'un texte. Mettez en place des métriques plus parlantes.

- **Perplexité (PPL) :** C'est la mesure standard en NLP. Elle représente, intuitivement, le nombre de choix hésitants du modèle à chaque pas.

$$PPL(X) = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log P(x_i|x_{<i})\right)$$

  Attention : la *CrossEntropyLoss* de PyTorch correspond déjà à la partie logarithmique (en base e).

- **Diversité Lexicale :** Un modèle "bête" a tendance à se répéter. Utilisez une métrique comme le ratio Type/Token (TTR) ou la librairie lexical-diversity pour mesurer la richesse du vocabulaire généré.
- **Taux d'hallucination (optionnel) :** Calculez la proportion de mots générés qui n'existent pas dans un dictionnaire de référence (ex: dictionnaire français ou vocabulaire du train set).

3 Améliorer la génération

C'est la partie la plus importante. Vous devez implémenter au moins l'une des améliorations suivantes pour rendre votre modèle plus performant ou plus pertinent.

3.1 Stratégie A : améliorer le sampling (top-k / Nucleus)

La génération par défaut est souvent trop aléatoire ou trop déterministe. Implémentez :

- **Température :** Divisez les logits avant le softmax pour lisser ou accentuer les probabilités.
- **Top-k sampling :** On ne tire au sort que parmi les k tokens les plus probables (les autres sont mis à $-\infty$).
- **Top-p (Nucleus) sampling :** On tire parmi le plus petit ensemble de tokens dont la somme des probabilités dépasse un seuil p (ex: 0.9). C'est la méthode utilisée par GPT-4.

3.2 Stratégie B : recherche en faisceau (*beam search*)

Au lieu de générer token par token (approche "gloutonne"), le beam search explore k pistes parallèles et garde la meilleure séquence globale à la fin. Bien que ce soit coûteux, cela produit des phrases grammaticalement bien meilleures. Voir par ex. le post de M. Pain.

3.3 Stratégie C : changer de tokenization (BPE)

Le modèle "caractère par caractère" est limité. Les LLM modernes utilisent le *Byte Pair Encoding* (BPE).

- Utilisez une librairie comme `tiktoken` (OpenAI) ou `tokenizers` (HuggingFace) pour encoder votre texte en sous-mots, en veillant à bien contrôler la taille de vocabulaire pour que cela reste gérable avec vos ressources de calcul.
- Ré-entraînez votre Transformer sur ces nouveaux indices.
- Comparez la perte et la qualité du texte avec la version caractères.

  Attention : la perplexité ne se compare pas directement si le vocabulaire change.

4 Rendu et consignes

Le travail est à réaliser en binôme.
Date limite : **2 mars 2026**.
Les livrables sont à déposer sur la page moodle du cours :

- **Le Code :** Un notebook ou des scripts Python propres. L'implémentation du modèle doit rester lisible. Evitez les boîtes noires type `AutoModel` de HuggingFace car le but est de manipuler les tenseurs.
- **Le Rapport (2 à 4 pages) :**
  - Ne mettez pas de code (ou très peu).
  - Montrez des comparaisons entre différentes version des modèles, basées sur les mesures quantitatives.
  - Vous pouvez donner quelques exemples de textes générés (avant/après amélioration).
  - Expliquez vos choix : pourquoi le beam search a-t-il amélioré la cohérence ? Pourquoi le top-k évite-t-il les répétitions ?

Conseil : Commencez petit (petit modèle, petit corpus) pour itérer vite. Ne lancez le gros entraînement que pour la version finale. L'intérêt principal étant de *comprendre* comment ces architectures fonctionnent, quelles sont leurs forces, leurs limites..., évitez au maximum d'utiliser des outils de codage automatique et le recours à des LLMs, sauf peut-être pour vous expliquer vos erreurs.